Discipline : Enseignement de spécialité NSI

La programmation orientée objet: Les classes

Contenus : Vocabulaire de la programmation objet : classes, attributs, méthodes, objets.

Capacités attendus: Écrire la définition d'une classe. Accéder aux attributs et méthodes d'une classe.

Pré-requis : Programmation Python

La programmation orientée objet

La programmation orientée objet (ou POO en abrégé) correspond à une autre manière d'imaginer, de construire et d'organiser son code.

La programmation orientée objet repose sur le concept d'objets qui sont des entités qui vont pouvoir posséder un ensemble de variables et de fonctions qui leur sont propres.

Les objectifs principaux de la programmation orientée objet sont de nous permettre de créer des scripts plus clairs, mieux structurés, plus modulables et plus faciles à maintenir et à déboguer.

Dans la vie réelle, un objet possède des caractéristiques et nous permet de réaliser des actions. Un crayon par exemple possède une taille, une couleur, une forme, etc. qui sont ses caractéristiques et permet d'écrire ou de dessiner.

Le concept d'objets en informatique s'inspire fortement de cette définition de la vie réelle : on va appeler "objet" un bloc cohérent de code qui possède ses propres variables (qui sont l'équivalent des caractéristiques des objets de tous les jours) et fonctions (qui sont nos actions). Comme les objets de la vie courante, les objets informatiques peuvent être très simples ou très complexes.

A Vocabulaire en programmation orientée objet

Dans la plupart des langages informatiques, on parle plutôt de "membres" pour désigner les variables et fonctions d'un objet et d'attributs pour désigner les variables et de "méthodes" pour désigner les fonctions.

Une classe est équivalente à un nouveau type de données. On connaît déjà par exemple les classes list ou str et les nombreuses méthodes permettant de les manipuler, par exemple :

```
[3, 5, 1].sort()
casse.upper()
```

Un objet ou une instance est un exemplaire particulier d'une classe. Par exemple [3, 5, 1] est une instance de la classe list et casse est une instance de la classe str.

Les objets sont caractérisés par deux éléments : les données nommées simplement attributs et les fonctions applicables appelées méthodes.

Par exemple un objet de la classe rectangle possède :

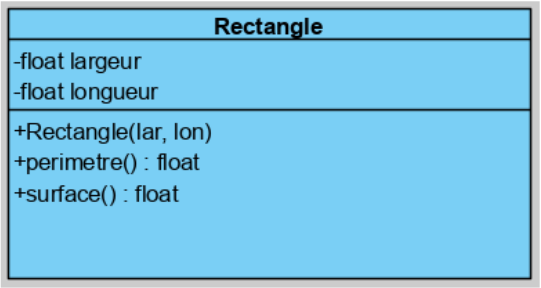
deux attributs : largeur et longueur ;
plusieurs méthodes, comme perimetre(), surface(), ..

La plupart des classes encapsulent à la fois les attributs et les méthodes applicables aux objets. Par exemple un objet str contient une chaîne de caractères Unicode (les données) et de nombreuses méthodes.

B Définition d'une classe

Une **classe** d'objets décrit un groupe d'objets ayant des propriétés similaires (**attributs**), un comportement commun (**méthodes**). Il s'agit en résumé d'un modèle d'objet.

Chaque objet est dit **instance** de sa classe. Chaque instance partage des noms d'attributs et d'opérations ou méthodes avec les autres instances de sa classe.
Le diagramme de classes permet de décrire sa structure finale. La recherche de ce diagramme doit être effectuée avant toute programmation.



Pour déclarer une classe en Python :
class Rectangle :

<u>B-1 Définition des attributs de la classe</u>
Pour créer un attribut qui sera reconnus dans toutes les méthodes de la classe, il suffit de les déclarer au début de la classe. Il existe deux types d'attributs publics et privés. Les attributs privés ne sont accessibles que dans la définition des méthodes de la classe contrairement aux attributs publics.

<u>Déclaration d'attribut public :</u>

```
class Rectangle :
        largeur=0.0
        longueur=0.0
```

Cette écriture permet de fixer le type ici ***float***.

Dans le main :
a=Rectangle()
a.largeur
L'attribut largeur est accessible à l'extérieur de la classe via l'objet instancié.

<u>Déclaration d'attribut privé :</u>

```
class Rectangle :
        __largeur=0.0
        __longueur=0.0
```

Dans le main :
a=Rectangle()
a.__largeur # renvoie une erreur

L'attribut largeur n'est pas accessible à l'extérieur de la classe via l'objet instancié car il est déclaré privé. Pour y accéder en lecteur ou écriture, on créé des méthodes publics (accesseurs) qui commence par « set » ou « get ».
Pour lire un attribut privé :

getLargeur() qui renvoit la valeur de __largeur

```
def getLargeur(self) :
    return  self.__largeur
```

setLargeur(valeur) qui modifie la valeur de __largeur

```
def setLargeur(self,valeur) :
    self.__largeur=valeur
```

B-2 Définition d'un constructeur

Le constructeur d'une classe est une méthode particulière au même nom que la classe qui sert **à initialiser** les propriétés de **l'objet instancié**. Ici le constructeur prend en argument deux variables ***lar*** et ***lon*** qui fixeront les valeurs des attributs ***largeur*** et ***longueur*** de la classe ***Rectangle***.

```
class Rectangle :
    def __init__(self,lar, lon):
        self.largeur=lar
        self.longueur=lon
```

Dans notre exemple, on définit un constructeur qui initialisera les attributs ***largeur*** et ***longueur*** de la classe ***Rectangle***. Le mot-clé ***self*** indique l'appartenance des éléments à la classe en cours de définition.

B-3 Définition des méthodes

Après avoir défini le constructeur, les méthodes de la classe peuvent être définies.

```
class Rectangle :
    def perimetre(self):
        return (self.largeur+self.longueur)*2
```

Exercice : Déclarer la méthode ***surface()***

```
class Rectangle :
    def surface(self):
        return self.largeur*self.longueur
```

C Utilisation de la classe

Une fois la classe ***Rectangle*** définie, on peut l'utiliser en instanciant un objet à partir du modèle. Cet objet

est créé dans un nouveau fichier qu'on appellera le programme principal (main).

```
if __name__ == '__main__':
        a=Rectangle(4,5) # appel du constructeur de la classe Rectangle
        print (str(a.perimetre()) + «  m »)
        print(str(a.surface()) + « m2 »)
```

Pour utiliser une classe provenant d'un autre fichier python se trouvant dans le même dossier, on écrit dans la partie déclaration au début du fichier contenant le programme principal :

from Rectangle import Rectangle

Note :
En PHP, C++, java, javascript, qui sont eux aussi des langages orientés objet, on peut instancier des objets dynamiquement à l'aide de la commande ***new*** (n'existe pas en langage Python). La différence, c'est que l'allocation de mémoire se fait sur la mémoire de masse ce qui impose de restituer cet espace mémoire à la destruction de l'objet. Ces langages font appel à des destructeurs pour libérer la mémoire allouée à la construction de l'objet. L'accès aux méthodes dans ce cas n'est plus le point mais la flèche.

Exemple en PHP :

```
$a=new PDO(« localhost », »root », » », »parc_info ») ;
$a->query($requete) ;
```

On accède à la méthode query() de la classe PDO en utilisant la flèche.

Exercice : Classe Parallelepipede

- Créer une classe ***Parallelepipede*** prenant en argument un objet de type Rectangle et dotée en plus d'un attribut ***hauteur*** et d'une autre méthode ***Volume()*** permettant de calculer le volume du Parallélépipède.
- Instancier un objet dans le programme principal pour tester la méthode ***Volume()***.